

1. Принцип работы Tools

Tools — это единственный способ бота совершать реальные действия. LLM формирует вызов инструмента в структурированном виде, backend его выполняет и возвращает результат.

```
// Общий формат вызова tool ToolCall { tool_name: string parameters: Record<string, any> called_by_skill: SkillName conversation_id: string }
```

2. Контракты инструментов

search_knowledge_base()

RAG-поиск по базе знаний салона. Возвращает релевантные фрагменты для формирования ответа.

Input	<pre>query: string // поисковый запрос filters?: { category?: "services" "prices" "contraindications" "faq" "aftercare" service_id?: string } top_k?: number // кол-во результатов (default: 5)</pre>
Output	<pre>results: [{ chunk: string, score: float, source: string }]</pre>
Errors	<pre>KNOWLEDGE_BASE_UNAVAILABLE — VectorDB недоступен NO_RESULTS_FOUND — нет релевантных результатов (confidence < 0.6)</pre>

get_service_info()

Получить полную информацию об услуге по ID или названию.

Input	<pre>service_id?: string service_name?: string // хотя бы один из двух</pre>
Output	<pre>service: { id, name, description, duration_min: int, price: float, contraindications: string[], preparation: string, aftercare: string, available_masters: string[] }</pre>
Errors	<pre>SERVICE_NOT_FOUND</pre>

check_available_slots()

Проверить доступные окна для записи через YClients API.

Input	<code>service_id: string</code> <code>master_id?: string</code> // если выбран мастер <code>date_from: ISO8601</code> <code>date_to: ISO8601</code>
Output	<code>slots: [</code> <code>{ datetime: ISO8601, master_id: string,</code> <code>master_name: string, duration_min: int }</code> <code>]</code>
Errors	<code>BOOKING_API_UNAVAILABLE</code> <code>NO_SLOTS_AVAILABLE</code> — нет свободных окон в диапазоне

create_booking()

Создать запись в YClients. Требуется явного подтверждения от клиента.

Input	<code>client_id: string</code> <code>service_id: string</code> <code>master_id: string</code> <code>datetime: ISO8601</code> <code>client_phone: string</code> <code>notes?: string</code>
Output	<code>booking: {</code> <code>booking_id: string,</code> <code>confirmation_code: string,</code> <code>datetime: ISO8601,</code> <code>master_name: string,</code> <code>address: string</code> <code>}</code>
Errors	<code>SLOT_ALREADY_TAKEN</code> — слот занят (нужно предложить другой) <code>CLIENT_NOT_FOUND</code> <code>BOOKING_API_ERROR</code>

reschedule_booking()

Перенести существующую запись.

Input	<code>booking_id: string</code> <code>new_datetime: ISO8601</code> <code>reason?: string</code>
Output	<code>updated_booking: { booking_id, new_datetime, status: "rescheduled" }</code>
Errors	<code>BOOKING_NOT_FOUND</code> <code>SLOT_UNAVAILABLE</code> <code>RESCHEDULE_NOT_ALLOWED</code> — менее 2 часов до записи

cancel_booking()

Отменить запись.

Input	booking_id: string reason?: string cancelled_by: "client" "admin"
Output	result: { booking_id, status: "cancelled", refund_info?: string }
Errors	BOOKING_NOT_FOUND CANCEL_NOT_ALLOWED — нарушает политику отмены

get_client_profile()

Получить профиль клиента из CRM.

Input	user_id: string
Output	profile: { id, name, phone, created_at, preferred_time_slots: string[], favorite_services: string[], last_visit: ISO8601, visit_count: int, contraindications: string[], course_stage?: { course_id, current_session, total_sessions } }
Errors	CLIENT_NOT_FOUND → вернуть пустой профиль, не ошибку

save_food_log()

Сохранить запись о еде (из фото или текста).

Input	user_id: string image_url?: string // если фото food_text?: string // если текст timestamp: ISO8601
Output	food_entry: { entry_id, detected_items: string[], calories: int, protein_g: float, fat_g: float, carbs_g: float, confidence: float }
Errors	FOOD_RECOGNITION_FAILED — не удалось распознать (confidence < 0.5) STORAGE_ERROR

save_water_log()

Залогировать выпитую воду.

Input	user_id: string amount_ml: int timestamp: ISO8601
Output	result: { entry_id, total_today_ml: int, daily_goal_ml: int, progress_pct: float }

Errors	STORAGE_ERROR
---------------	---------------

send_notification()

Отправить уведомление клиенту или администратору.

Input	recipient_id: string recipient_type: "client" "admin" channel: "telegram" "whatsapp" "sms" template: string variables: Record<string, string> schedule_at?: ISO8601 // отложенная отправка
Output	result: { notification_id, status: "sent" "scheduled", sent_at: ISO8601 }
Errors	CHANNEL_UNAVAILABLE TEMPLATE_NOT_FOUND RECIPIENT_NOT_FOUND

create_admin_task()

Создать задачу для администратора (хэндофф, нестандартный запрос, жалоба).

Input	type: "handoff" "complaint" "custom_request" "medical_risk" priority: "low" "medium" "high" "urgent" client_id: string conversation_id: string description: string context_summary: string
Output	task: { task_id, assigned_to?: string, status: "open", created_at: ISO8601 }
Errors	ADMIN_SYSTEM_UNAVAILABLE

handoff_to_human()

Официально передать диалог человеку. Переводит сессию в статус HUMAN_HANDOFF.

Input	conversation_id: string reason: string priority: "normal" "urgent" create_task: boolean
Output	result: { status: "handed_off", task_id: string, estimated_response_min: int }
Errors	NO_ADMIN_AVAILABLE — вернуть estimated_response_time

3. Карта интеграций

Система	Протокол	Используется для
YClients	REST API + Webhook	Слоты, записи, мастера, клиенты
VectorDB (Qdrant)	gRPC / REST	RAG-поиск по базе знаний
PostgreSQL	TCP / ORM	Профили клиентов, история, логи
Redis	TCP	Диалоговая память, сессии, rate limiting
CRM (внутренний)	REST API	Профили, сегментация, события
Food Recognition API	REST + multipart	Распознавание еды по фото
Notification Service	REST / Queue	Push, SMS, мессенджеры
Analytics Service	Event stream / REST	Event tracking, метрики